«Код Будущего — программирование на Python для специалистов по информационной безопасности»

Модуль 2. Файлы, данные, простая обработка и визуализация.

ТЕМА Визуализация данных (библиотека matplotlib)

Практическое занятие

Цель:

Изучить способы графического отображения информации.

Задачи:

Узнаем как рисовать графики

Какие есть графики.

Методические указания и инструкции по выполнению задания

1. **Вспомните и повторите теорию, необходимую для выполнения практического задания**

Ключевые моменты:

Matplotlib — основная библиотека Python для визуализации данных. Позволяет строить:

Линейные графики (plt.plot).

Столбчатые диаграммы (plt.bar).

Круговые диаграммы (plt.pie).

Точечные графики (plt.scatter).

Гистограммы (plt.hist).

Примеры использования:

Анализ временных рядов (логи сервера).

Сравнение категориальных данных (типы уязвимостей).

Дополнительные возможности:

Размещение нескольких графиков в одной фигуре (subplots).

Графики распределения (boxplot, kdeplot).

2. **Разберитесь в том, как выполняется задание**

Пример программы для построения графика с помощью Matplotlib.

Задание 1

Визуализация статистики посещений сайта (анализ трафика)

Цель: выявить подозрительную активность (DDoS, сканеры уязвимостей, брутфорс)

Описание для тренажера:

1. Реализовать функцию, которая анализирует список посещений сайта.

2. Функция принимает два параметра: список IP-адресов `logs` и порог подозрительных запросов `limit` (по умолчанию 100).

3. Проверить, что `logs` — это список. Если нет, вызвать ошибку `ValueError`.

4. Подсчитать, сколько раз каждый IP встречается в списке.

5. Игнорировать значения, которые не являются строками.

6. Вернуть список IP-адресов, количество которых превышает порог `limit`.

7. Не использовать запрещённые конструкции и модули: os, sys, subprocess, socket, importlib, eval, exec, compile, open.

Тренажер - 20701-website-visit-statistics

Решение:

```
SAFE_REQUEST_LIMIT = 100  # больше 100 запросов — подозрительно


def analyze_traffic(logs, limit=SAFE_REQUEST_LIMIT):
    """
    Анализирует список посещений сайта и возвращает список IP,
    которые превысили заданный лимит запросов.

    :param logs: list[str] — список IP-адресов
    :param limit: int — порог подозрительных запросов
    :return: list[str] — список подозрительных IP
```

```
"""

if not isinstance(logs, list):
    raise ValueError("Logs must be a list")

counter = {}

for ip in logs:
    if not isinstance(ip, str):
        continue
    counter[ip] = counter.get(ip, 0) + 1

return [ip for ip, count in counter.items() if count > limit]
```

Задание 2
Визуализация событий безопасности (SIEM)

Цель: анализ срабатываний IDS/IPS

Описание для тренажера:

1. Реализовать функцию, которая анализирует список событий безопасности.

2. Функция принимает два параметра: список событий `events` и порог критичности `threshold` (по умолчанию 7).

3. Проверить, что `events` — это список. Если нет, вызвать ошибку `ValueError`.

4. Игнорировать элементы списка, которые не являются словарями.

5. Для каждого словаря проверить поле `severity`. Если `severity` — число и оно больше или равно `threshold`, добавить событие в список критических.

6. Вернуть список критических событий.

7. Не использовать запрещённые конструкции и модули: os, sys, subprocess, socket, importlib, eval, exec, compile, open.

Тренажер - 20702-security-events

Решение:

```
# index.py


CRITICAL_SEVERITY = 7  # события с severity >= 7 считаются критическими



def analyze_security_events(events, threshold=CRITICAL_SEVERITY):
    if not isinstance(events, list):
        raise ValueError("Events must be a list")

    critical_events = []

    for event in events:
        if not isinstance(event, dict):
            continue
        severity = event.get("severity")
        if isinstance(severity, (int, float)) and severity >= threshold:
            critical_events.append(event)

    return critical_events
```

Задание 3

Проведите анализ геолокации атак (картографическая визуализация с matplotlib)

Описание для тренажера

1. Реализовать функцию для визуализации геолокации атак.

2. Функция принимает один параметр: путь к CSV-файлу `input_file` (по умолчанию "attacks.csv").

3. Прочитать CSV-файл с помощью pandas.

4. Проверить, что в файле есть столбцы "Широта" и "Долгота". Если их нет, вывести сообщение: Файл должен содержать столбцы 'Широта' и 'Долгота'.

5. Построить график точечной диаграммы с matplotlib, где ось X — Долгота, ось Y — Широта, точки красного цвета с прозрачностью 0.5.

6. Добавить название графика "Геолокация атак", подписи осей и сетку.

7. Сохранить график в файл "attack_map.png" и вывести сообщение: Карта сохранена как attack_map.png.

8. Если файла CSV нет, вывести сообщение: Файл {input_file} не найден.

9. Не использовать запрещённые конструкции и модули: sys, subprocess, socket, importlib, builtins, eval, exec, compile.

Тренажер - 20703-geolocation-analysis-of-attacks

Пример корректного решения:

```
import matplotlib.pyplot as plt
import pandas as pd
def plot_attack_locations(input_file="attacks.csv"):
    Визуализирует геолокацию атак на карте
    try:
        df = pd.read_csv(input_file)

        plt.figure(figsize=(10, 6))
        plt.scatter(df['Долгота'], df['Широта'], alpha=0.5, c='red')
        plt.title("Геолокация атак")
        plt.xlabel("Долгота")
        plt.ylabel("Широта")
        plt.grid(True)
        plt.savefig("attack_map.png")
        print("Карта сохранена как attack_map.png")

    except FileNotFoundError:
        print(f"Файл {input_file} не найден")
    except KeyError:
        print("Файл должен содержать столбцы 'Широта' и 'Долгота'")
# Пример использования
plot_attack_locations()
```

Задание 4

Временная шкала инцидентов

Описание для тренажера

1. Реализовать функцию для визуализации временной шкалы инцидентов.

2. Функция принимает один параметр: путь к CSV-файлу `input_file` (по умолчанию "incidents.csv").

3. Прочитать CSV-файл с помощью pandas.

4. Проверить, что файл содержит столбцы "timestamp" и "incident_type". Если нет, вывести сообщение: Файл должен содержать столбцы 'timestamp' и 'incident_type'.

5. Преобразовать колонку "timestamp" в формат даты/времени. Если преобразовать не удалось, вывести сообщение: Не удалось преобразовать 'timestamp' в формат даты/времени.

6. Построить график, где по оси X — дата, по оси Y — количество инцидентов в этот день. Использовать столбчатую диаграмму с синим цветом и прозрачностью 0.7.

7. Добавить заголовок графика "Временная шкала инцидентов", подписи осей и сетку.

8. Сохранить график в файл "incident_timeline.png" и вывести сообщение: График сохранен как incident_timeline.png.

9. Если файла CSV нет, вывести сообщение: Файл {input_file} не найден.

10. Не использовать запрещённые конструкции и модули: sys, subprocess, socket, importlib, builtins, eval, exec, compile.

Тренажер - 20704-Incident-Timeline

Решение:

```
# index.py

import pandas as pd
```

```
def plot_incident_timeline(input_file="incidents.csv"):
    """
    Визуализирует временную шкалу инцидентов.
    :param input_file: str — CSV файл с колонками 'timestamp' и 'incident_type'
    """
    try:
        df = pd.read_csv(input_file)
    except FileNotFoundError:
        print(f"Файл {input_file} не найден")
        return

    # Проверка колонок
    if "timestamp" not in df.columns or "incident_type" not in df.columns:
        print("Файл должен содержать столбцы 'timestamp' и 'incident_type'")
        return

    # Преобразуем колонку timestamp в datetime
    try:
        df["timestamp"] = pd.to_datetime(df["timestamp"])
    except Exception:
        print("Не удалось преобразовать 'timestamp' в формат даты/времени")
        return

    # Попытка построить график
    try:
        import matplotlib.pyplot as plt

        plt.figure(figsize=(10, 6))
        df.groupby(df["timestamp"].dt.date).size().plot(kind='bar', color='blue', alpha=0.7)
        plt.title("Временная шкала инцидентов")
        plt.xlabel("Дата")
        plt.ylabel("Количество инцидентов")
        plt.grid(True)
        plt.tight_layout()
        plt.savefig("incident_timeline.png")
        plt.close()
        print("График сохранен как incident_timeline.png")
    except ImportError:
        print("matplotlib не установлен, график не построен")
```

Задание 5

Heatmap подозрительной активности

Описание для тренажера

1. Реализовать функцию для построения тепловой карты подозрительной активности.

2. Функция принимает один параметр: путь к CSV-файлу `input_file` (по умолчанию "activity.csv").

3. Прочитать CSV-файл с помощью pandas.

4. Проверить, что файл содержит столбцы "x" и "y". Если нет, вывести сообщение: Файл должен содержать столбцы 'x' и 'y'.

5. Построить тепловую карту с помощью seaborn (`sns.kdeplot`) с заполнением (`fill=True`), цветовой схемой "Reds" и прозрачностью 0.7.

6. Настроить размер графика 8x6, добавить заголовок "Heatmap подозрительной активности" и подписи осей "X" и "Y".

7. Сохранить график в файл "suspicious_heatmap.png" и вывести сообщение: Heatmap сохранена как suspicious_heatmap.png.

8. Если файла CSV нет, вывести сообщение: Файл {input_file} не найден.

9. Если не установлены matplotlib или seaborn, вывести сообщение: matplotlib или seaborn не установлены, график не построен.

10. Не использовать запрещённые конструкции и модули: sys, subprocess, socket, importlib, builtins, eval, exec, compile.

Тренажер - 20705-Heatmap-of-Suspicious-Activity

Решение:

```
# index.py


import pandas as pd
```

```
def plot_suspicious_heatmap(input_file="activity.csv"):
    """
    Строит тепловую карту подозрительной активности.
    :param input_file: str — CSV с колонками 'x' и 'y'
    """
    try:
        df = pd.read_csv(input_file)
    except FileNotFoundError:
        print(f"Файл {input_file} не найден")
        return

    # Проверка колонок
    if "x" not in df.columns or "y" not in df.columns:
        print("Файл должен содержать столбцы 'x' и 'y'")
        return

    # Попытка построить график
    try:
        import matplotlib.pyplot as plt
        import seaborn as sns

        plt.figure(figsize=(8, 6))
        sns.kdeplot(x=df['x'], y=df['y'], fill=True, cmap="Reds", alpha=0.7)
        plt.title("Heatmap подозрительной активности")
        plt.xlabel("X")
```

```
    plt.ylabel("Y")
    plt.tight_layout()
    plt.savefig("suspicious_heatmap.png")
    plt.close()
    print("Heatmap сохранена как suspicious_heatmap.png")
except ImportError:
    print("matplotlib или seaborn не установлены, график не построен")
```

Критерии оценивания

Критерий	**Баллы**	**Требования**
1. Корректность работы программы	0-1	– 1 балл: программа работает без ошибок, полностью соответствует условиям задачи, и выводит правильный ответ. – 0 баллов: программа не работает или выдает критичные ошибки, выводит неправильный ответ.
2. Использование необходимых функций	0-2	– 2 балла: при решении использованы все требуемые необходимые языковые средства – 1 балл: при решении использованы не все необходимые языковые средства – 0 баллов: требуемые языковые средства не использованы

Шкала оценивания:

Зачтено — 3 балла

Не зачтено — 0 – 2 балла

Требования к оформлению результата выполнения задания: решение задачи предусматривает ввод программного кода в специальное поле тренажера